

Application: Cloud Data Backup System

Given Recurrence

$$T(n) = 2T(n/2) + n \log n \quad T(n) = 2T(n/2) + n \log n \quad T(n) = 2T(n/2) + n \log n$$

Applying the Master Theorem

The recurrence is of the form:

$$T(n) = aT(n/b) + f(n) \quad T(n) = aT(n/b) + f(n) \quad T(n) = aT(n/b) + f(n)$$

where,

- $a = 2$
- $b = 2$
- $f(n) = n \log n$

Calculate:

$$n \log_b a = n \log_2 2 = n \cdot 1 = n \quad n^{\log_b a} = n^1 = n \quad n \log_b a = n \log_2 2 = n$$

Compare:

- $f(n) = n \log n$
- $n \log_b a = n$

Since

$$f(n) = \Theta(n \log n) = \Theta(n \log_b a \log n) \quad f(n) = \Theta(n \log n) = \Theta(n \log_b a \log n) \\ = \Theta(n^{\log_b a} \log n) \quad f(n) = \Theta(n \log n) = \Theta(n \log_b a \log n)$$

this matches **Master Theorem Case 2**, where

$$f(n) = \Theta(n \log_b a \log^k n), k=1 \quad f(n) = \Theta(n \log_b a \log^k n), k=1 \\ k=1 \quad f(n) = \Theta(n \log_b a \log^k n), k=1$$

Therefore,

$$T(n) = \Theta(n \log^{k+1} n) \quad T(n) = \Theta(n \log^{k+1} n) \quad T(n) = \Theta(n \log^{k+1} n) \\ T(n) = \Theta(n (\log n)^2) \quad T(n) = \Theta(n (\log n)^2) \quad T(n) = \Theta(n (\log n)^2)$$

Case Identification

Master Theorem: Case 2

Because

$$f(n) = \Theta(n \log^2 n) = \Theta(n \log^2 n) \quad f(n) = \Theta(n \log^2 n) = \Theta(n \log^2 n)$$

Scalability Analysis

- The data is divided into **two equal halves** recursively.
- Each level performs **$n \log n$** work.
- There are **$\log n$** levels of recursion.
- Hence, the total running time grows as:

$$n \log n \times \log n = n (\log n)^2$$

As the data size increases, the algorithm scales efficiently but is slightly slower than $O(n \log n)$ because of the extra logarithmic factor.

Asymptotic Time Complexity

$$\Theta(n (\log n)^2)$$

Source Code

```
import math

def cloud_backup(n):
    if n <= 1:
        return 1
    return 2 * cloud_backup(n // 2) + n * math.log2(n)

n = int(input("Enter the data size: "))

result = cloud_backup(n)

print("Estimated backup work:", round(result, 2))
print("Time Complexity:  $\Theta(n (\log n)^2)$ ")
```

Output

```
import math

def cloud_backup(n):
    if n <= 1:
        return 1
    return 2 * cloud_backup(n // 2) + n * math.log2(n)

n = int(input("Enter the data size: "))

result = cloud_backup(n)

print("Estimated backup work:", round(result, 2))
print("Time Complexity:  $\Theta(n (\log n)^2)$ ")
```

```
Enter the data size: 8
Estimated backup work: 56.0
Time Complexity:  $\Theta(n (\log n)^2)$ 
```